



INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

14 May 2024

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 2040

SYSTEM PROGRAMMING

WEEK 13 LECTURE 1

EXCEPTIONAL CONTROL FLOW

EXCEPTIONAL CONTROL FLOW

LOADING AND RUNNING PROGRAMS

execve : Loading and Running Programs

- * The `execve` function loads and runs a new program in the context of the current process.

```
#include <unistd.h>
```

```
int execve(const char *filename, const char *argv[], const char *envp[]);
```

Does not return if OK, returns -1 on error

- The `execve` function loads and runs the executable object file `filename` with the argument list `argv` and the environment variable list `envp`.
- `Execve` returns to the calling program only if there is an error such as not being able to find `filename`.
- **So unlike `fork`, which is called once but returns twice, `execve` is called once and never returns.**

execve: Loading and Running Programs

- * `int execve(char *filename, char *argv[], char *envp[])`
- * Loads and runs in the current process:
 - Executable file **filename**
 - * Can be object file or script file beginning with `#!/interpreter` (e.g., `#!/bin/bash`)
 - ...with argument list **argv**
 - * By convention `argv[0]==filename`
 - ...and environment variable list **envp**
 - * “name=value” strings (e.g., `USER=droh`)
 - * `getenv`, `putenv`, `putenv`
- * Overwrites code, data, and stack
 - Retains PID, open files and signal context
- * Called **once** and **never** returns
 - ...except if there is an error

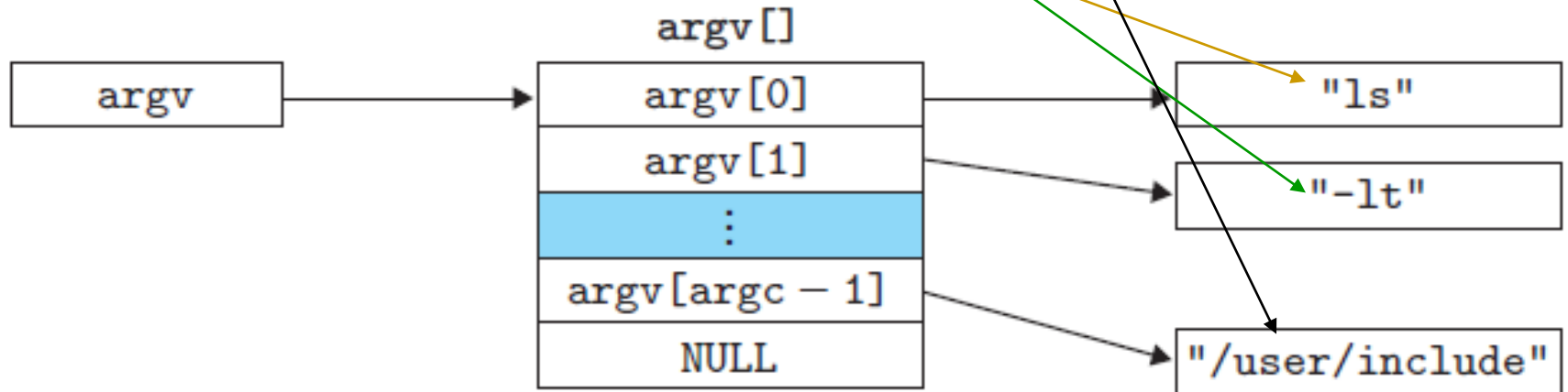
execve: Loading and Running Programs

- The argument list is represented by the data structure shown in Figure.
- The argv variable points to a null-terminated array of pointers, each of which points to an argument string.
- By convention, argv[0] is the name of the executable object file.
- The list of environment variables is represented by a similar data structure, shown in Figure.
- The envp variable points to a null-terminated array of pointers to environment variable strings, each of which is a name-value pair of the form "NAME=VALUE".

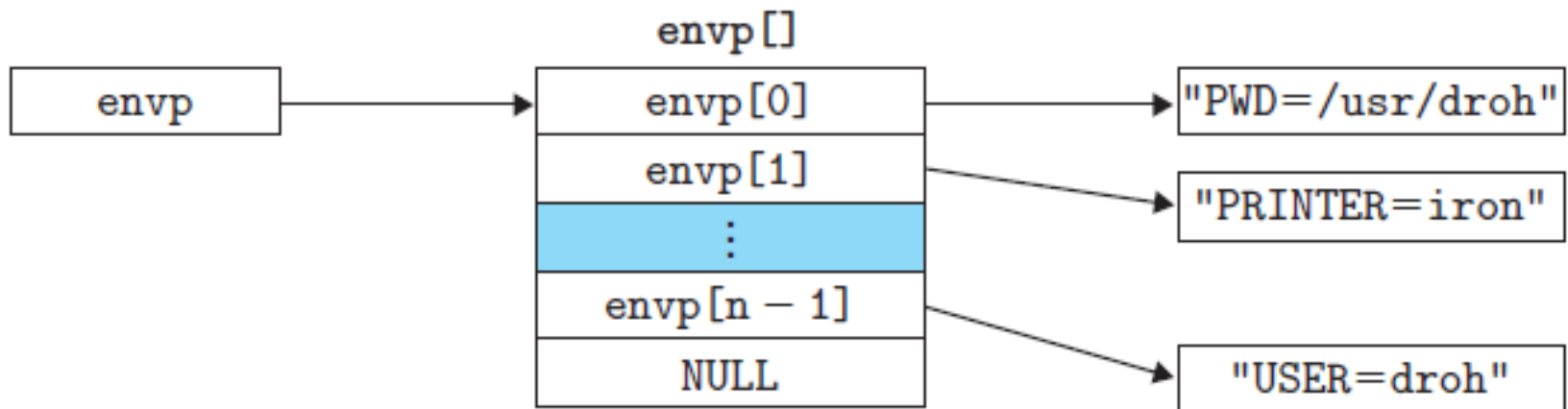
execve: Loading and Running Programs

Organization of an argument list.

```
$ls -lt /user/include
```



Organization of an environment variable list.



execve: Loading and Running Programs

- Unix provides several functions for manipulating the environment array:

```
#include <stdlib.h>
```

```
char *getenv(const char *name);
```

Returns: ptr to name if exists, NULL if no match

- The getenv function searches the environment array for a string “name=value”.
- If found, it returns a pointer to value, otherwise it returns NULL.

```
#include <stdlib.h>
```

```
int setenv(const char *name, const char *newvalue, int overwrite);
```

Returns: 0 on success, -1 on error

```
void unsetenv(const char *name);
```

Returns: nothing

- If the environment array contains a string of the form “name=oldvalue”, then unsetenv deletes it and setenv replaces oldvalue with newvalue, but only if overwrite is nonzero.
- If name does not exist, then setenv adds “name=newvalue” to the array.

execve : Loading and Running Programs

- Write a program called myecho that prints its command line arguments and environment variables.

```
Int main(int argc, char *argv[], char *envp[])
{
    int i;
    printf("Command-line arguments :\n");
    for(i=0; argv[i] != NULL; i++)
        printf("    argv[%2d] : %s\n", i, argv[i]);
    printf("\n");
    printf("Environment variables : \n");
    for(i=0; envp[i] != NULL; i++)
        printf("    envp[%2d] : %s\n", i, envp[i]);
    exit(0);
}
```

myecho.c

\$/myecho arg1 arg2

Command-line arguments:

argv[0]: myecho

argv[1]: arg1

argv[2]: arg2

Environment variables:

envp[0]: PWD=/usr0/droh/ics/code/ecf

envp[1]: TERM=emacs

...

envp[25]: USER=droh

envp[26]: SHELL=/usr/local/bin/tcsh

envp[27]: HOME=/usr0/droh

execve: Loading and Running Programs

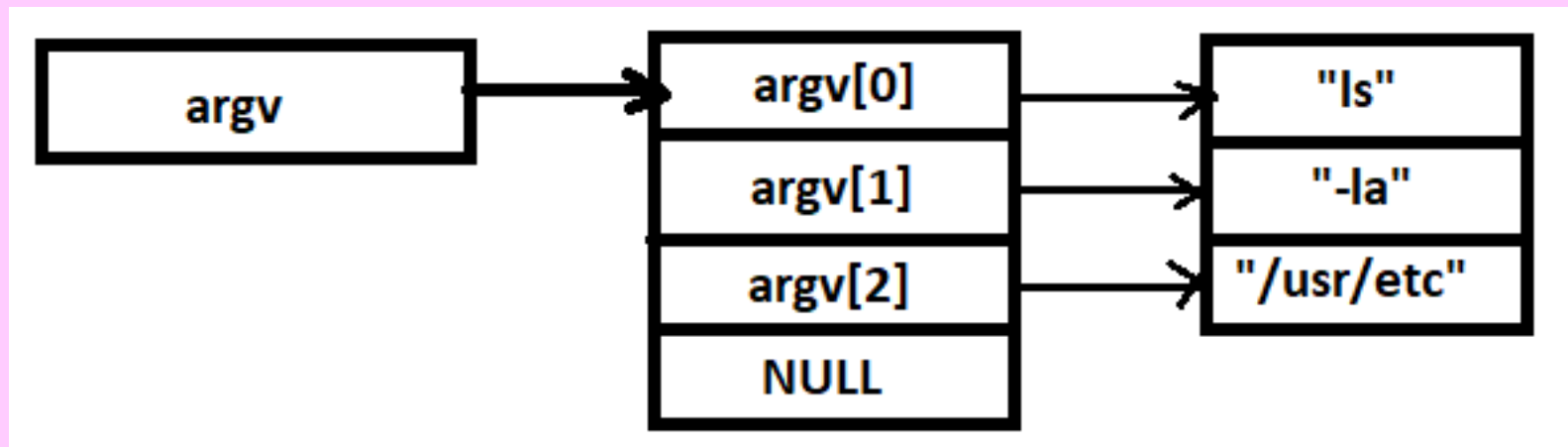
Consider the execution of the following command on linux:

```
$ls -la /usr/etc
```

This command line contains three arguments

```
*argv[0]="ls"    *argv[1]="-la"    *argv[2]="/usr/etc"
```

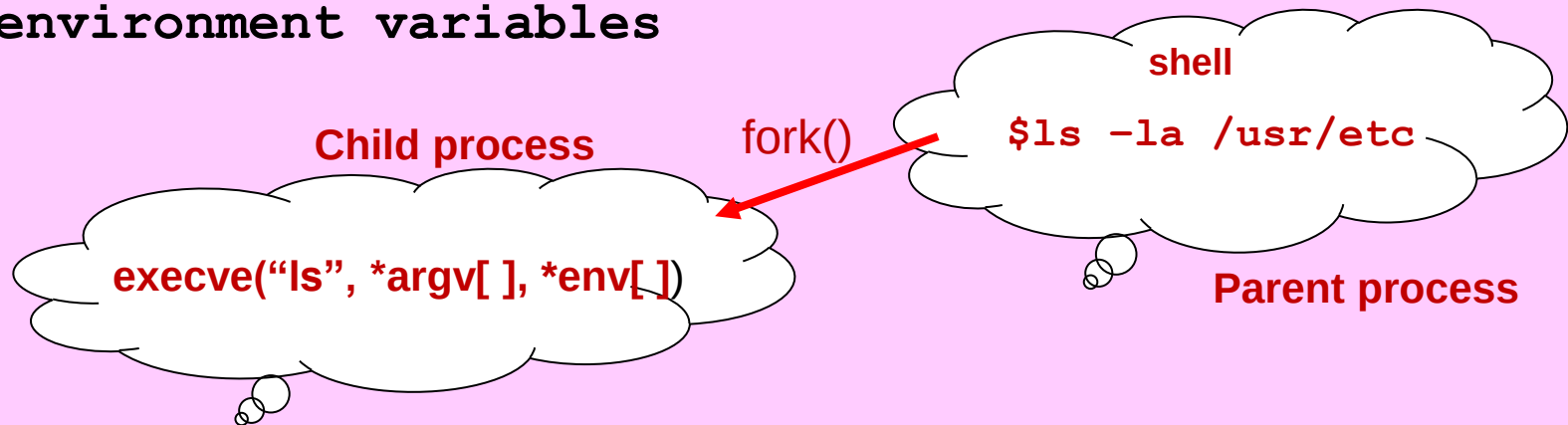
Here ls or argument 0 is the object code or command to be executed



execve: Loading and Running Programs

Consider the execution of a following a command on linux: `$ls -la /usr/etc`

- Here parent process is shell
- In order to execute the command `ls`, shell creates a child process by executing `fork` function
- The child process now invokes `execve` function to load the `ls` executable object code into memory and execute.
- The `argv` and `env[]` pointers vectors are stored in the stack along with the command line arguments and environment variables

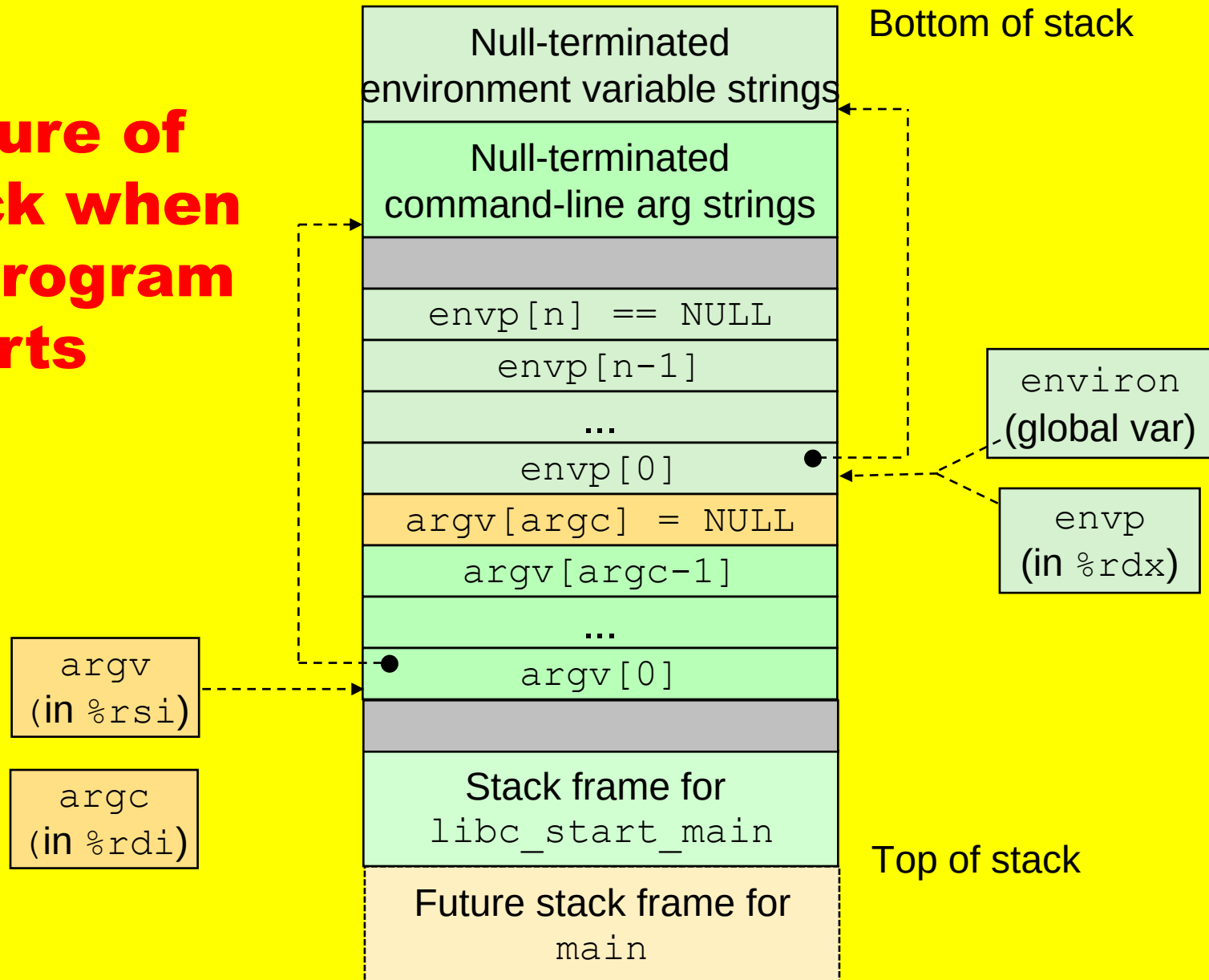


- After `execve` loads filename, it calls the startup code.
- The startup code sets up the stack and passes control to the main routine of the new program, which has a prototype of the form

```
int main(int argc, char **argv, char **envp);
```
- or equivalently,

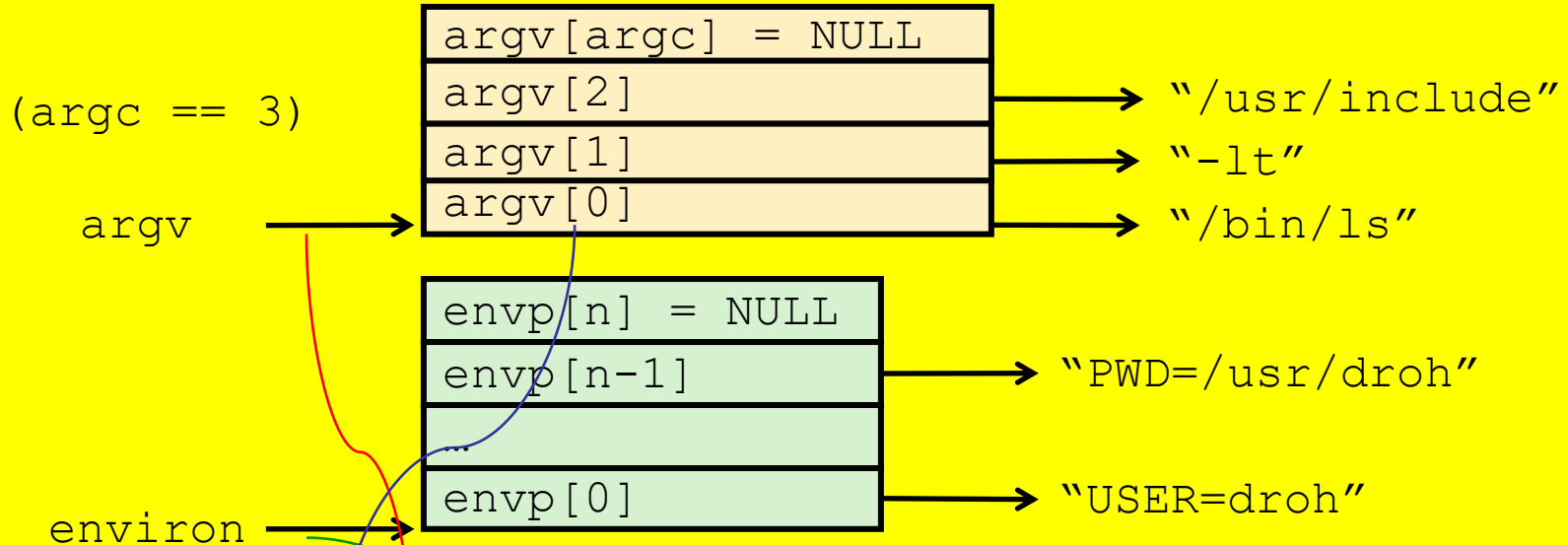
```
int main(int argc, char *argv[], char *envp[]);
```

Structure of the stack when a new program starts



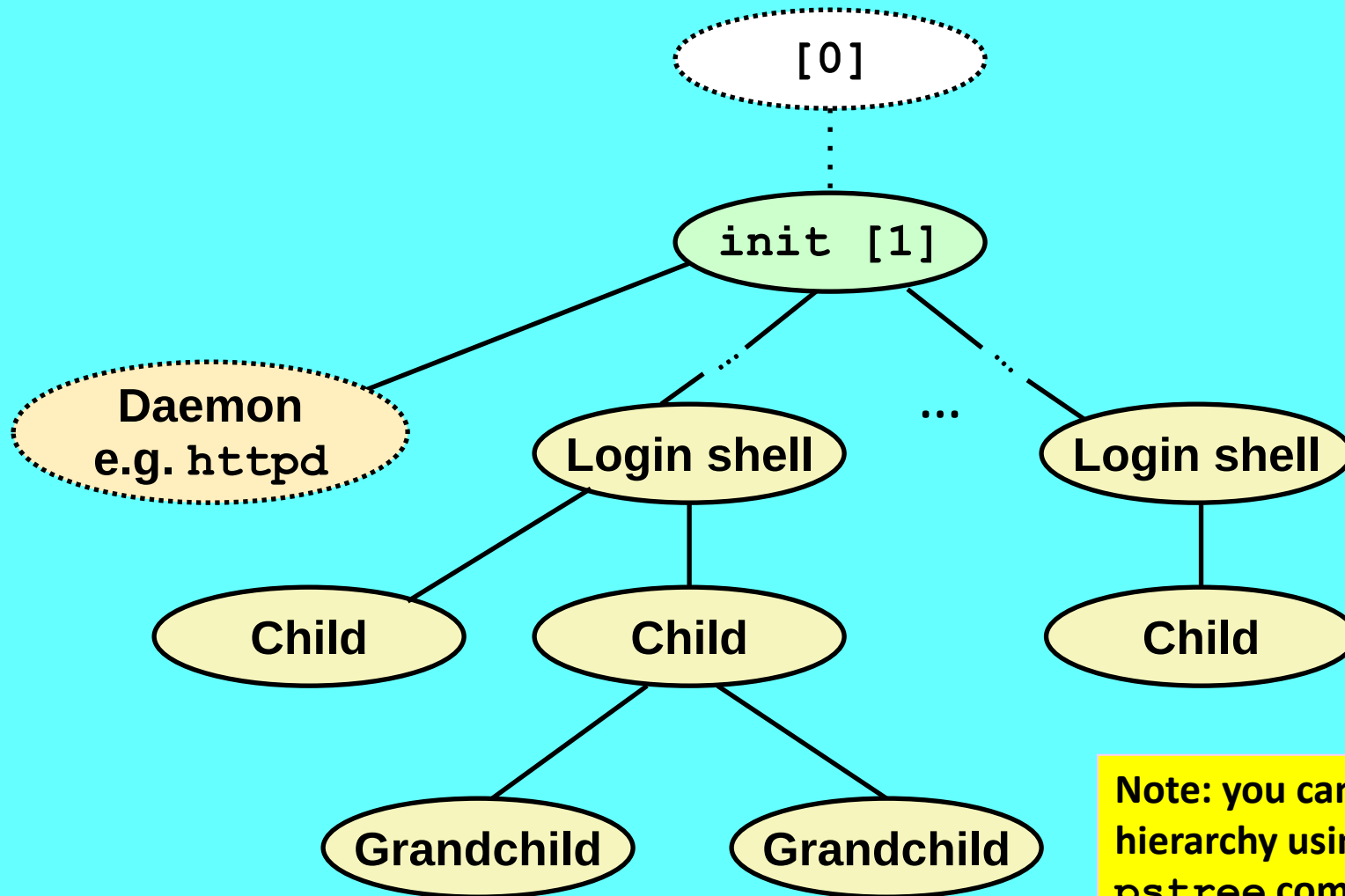
execve Example

- Executes `"/bin/ls -lt /usr/include"` in child process using current environment:



```
if ((pid = Fork()) == 0) { /* Child runs program */  
    if (execve(argv[0], argv, environ) < 0) {  
        printf("%s: Command not found.\n", myargv[0]);  
        exit(1);  
    }  
}
```

Linux Process Hierarchy



Note: you can view the hierarchy using the Linux `ps tree` command

Using fork and execve to Run Programs

- ★ Programs such as Unix shells and Web servers make heavy use of the fork and execve functions.
- ★ A shell is an interactive application-level program that runs other programs on behalf of the user.
- ★ The original shell was the sh program, which was followed by variants such as csh, tcsh, ksh, and bash.
- ★ A shell performs a sequence of read/evaluate steps, and then terminates.
- ★ The read step reads a command line from the user.
- ★ The evaluate step parses the command line and runs programs on behalf of the user.

Shell Programs

★ A **shell** is an application program that runs programs on behalf of the user.

- **sh** Original Unix shell (Stephen Bourne, AT&T Bell Labs, 1977)
- **csch/tcsch** BSD Unix C shell
- **bash** “Bourne-Again” Shell (default Linux shell)

```
int main()
{
    char cmdline[MAXLINE]; /* command line */

    while (1) {
        /* read */
        printf("> ");
        fgets(cmdline, MAXLINE, stdin);
        if (feof(stdin))
            exit(0);

        /* evaluate */
        eval(cmdline);
    }
}
```

shellcmdline.c

*Execution is a
sequence of
read/evaluate steps*

Shell Programs

- * Main routine of a simple shell is shown in figure below.
- * **The shell prints a command-line prompt, waits for the user to type a command line on stdin, and then evaluates the command line.**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <unistd.h>
#include <errno.h>
```

```
#define MAXARGS 128
#define MAXLINE 1024
```

```
/* Function prototypes */
```

```
void eval(char *cmdline);
int  parseline(char *buf, char **argv);
int  builtin_command(char **argv);
```

```
int main()
{
```

```
    char cmdline[MAXLINE]; /* Command line */
```

```
    while (1)
    {
```

```
        printf("> "); /* SHELL PROMPT */
```

```
        /* Read */
        fgets(cmdline, MAXLINE, stdin);
```

```
        if (feof(stdin))
            exit(0);
```

```
        /* Evaluate */
        eval(cmdline);
```

```
    }
```

```
}
```

The C library function `char *fgets(char *str, int n, FILE *stream)` reads a line from the specified stream and stores it into the string pointed to by `str`. It stops when either `(MAXLINE-1)` characters are read, the newline character is read, or the end-of-file is reached, whichever comes first.

`feof(stdin)` function returns a non-zero value when End-of-File indicator associated with the stream `stdin` is set, else zero is returned.

shellcmdline.c

Execution is a sequence of read/evaluate steps

```

/* eval - Evaluate a command line */
void eval(char *cmdline)
{
    /* Argument list execve() */
    char *argv[MAXARGS];
    extern char **environ;
    /* Holds modified command line */
    char buf[MAXLINE];

```

```

int bg; /* Should the job run in bg or fg? */
pid_t pid; /* Process id */

```

```

strcpy(buf, cmdline);
bg = parseline(buf, argv);

```

```

if (argv[0] == NULL)
    return; /* Ignore empty lines */
if (!builtin_command(argv))
{

```

```

    if ((pid = Fork()) == 0)
    { /* Child runs user job */
        if (execve(argv[0], argv, environ) < 0)
        {
            printf("%s: Command not found.\n", argv[0]);
            exit(0);
        }
    }
}

```

```

/* Parent waits for foreground job to terminate */

```

```

if (!bg)
{
    int status;
    if (waitpid(pid, &status, 0) < 0)
        unix_error("waitfg: waitpid error");
}
else
    printf("%d %s", pid, cmdline);
return;
}

```

shellcmdline.c

```

/* parseline - Parse the command line and build the argv array */
int parseline(char *buf, char **argv)

```

```

{
    char *delim; /* Points to first space delimiter */
    int argc; /* Number of args */
    int bg; /* Background job? */

```

```

/* Replace trailing '\n' with space */
buf[strlen(buf)-1] = ' ';

```

```

/* Ignore leading spaces */
while (*buf && (*buf == ' '))
    buf++;

```

```

/* Build the argv list */
argc = 0;

```

```

while ((delim = strchr(buf, ' '))
{
    argv[argc++] = buf;
    *delim = '\0';
    buf = delim + 1;
    /* Ignore spaces */
    while (*buf && (*buf == ' '))
        buf++;
}
argv[argc] = NULL;

```

```

if (argc == 0) /* Ignore blank line */
    return 1;

```

```

/* Should the job run in the background? */
if ((bg = (*argv[argc-1] == '&')) != 0)
    argv[--argc] = NULL;
return bg;

```

```

/* If first arg is a builtin command, run it and
return true */

```

```

int builtin_command(char **argv)
{
    if (!strcmp(argv[0], "quit")) /* quit command */
        exit(0);
    if (!strcmp(argv[0], "&")) /* Ignore singleton & */
        return 1;
    return 0; /* Not a builtin command */
}

```

ls -la /usr/bin

buf

delim

ls\0 -la\0 /usr/bin\0

argv[0]

argv[1]

argv[2]

```

/* eval - Evaluate a command line */
void eval(char *cmdline)
{
    /* Argument list execve() */
    char *argv[MAXARGS];
    extern char **environ;
    /* Holds modified command line */
    char buf[MAXLINE];

```

```

int bg; /* Should the job run in bg or fg? */
pid_t pid; /* Process id */

```

```

strcpy(buf, cmdline);
bg = parseline(buf, argv);

```

```

if (argv[0] == NULL)
    return; /* Ignore empty lines */
if (!builtin_command(argv))
{

```

```

    if ((pid = Fork()) == 0)
    { /* Child runs user job */
        if (execve(argv[0], argv, environ) < 0)
        {
            printf("%s: Command not found.\n", argv[0]);
            exit(0);
        }
    }
}

```

```

/* Parent waits for foreground job to terminate */

```

```

if (!bg)
{
    int status;
    if (waitpid(pid, &status, 0) < 0)
        unix_error("waitfg: waitpid error");
}
else
    printf("%d %s", pid, cmdline);
return;
}

```

shellcmdline.c

```

/* parseline - Parse the command line and build the argv array */
int parseline(char *buf, char **argv)
{

```

```

    char *delim; /* Points to first space delimiter */
    int argc; /* Number of args */
    int bg; /* Background job? */

```

```

/* Replace trailing '\n' with space */
buf[strlen(buf)-1] = ' ';

```

```

/* Ignore leading spaces */
while (*buf && (*buf == ' '))
    buf++;

```

```

/* Build the argv list */
argc = 0;

```

```

while ((delim = strchr(buf, ' '))
{
    argv[argc++] = buf;
    *delim = '\0';
    buf = delim + 1;
    /* Ignore spaces */
    while (*buf && (*buf == ' '))
        buf++;
}
argv[argc] = NULL;

```

```

if (argc == 0) /* Ignore blank line */
    return 1;

```

```

/* Should the job run in the background? */
if ((bg = (*argv[argc-1] == '&')) != 0)
    argv[--argc] = NULL;
return bg;

```

```

/* If first arg is a builtin GYU

```

```

if (!strcmp(argv[0], "quit")) /* quit command */
    exit(0);
if (!strcmp(argv[0], "&")) /* Ignore singleton & */
    return 1;
return 0; /* Not a builtin command */
}

```

ls -la /usr/bin

buf

delim

ls\0 -la\0 /usr/bin\0

argv[0]

argv[1]

argv[2]

```

/* eval - Evaluate a command line */
void eval(char *cmdline)
{
    /* Argument list execve() */
    char *argv[MAXARGS];
    extern char **environ;
    /* Holds modified command line */
    char buf[MAXLINE];

```

```

int bg; /* Should the job run in bg or fg? */
pid_t pid; /* Process id */

```

```

strcpy(buf, cmdline);
bg = parseline(buf, argv);

if (argv[0] == NULL)
    return; /* Ignore empty lines */
if (!builtin_command(argv))
{

```

```

    if ((pid = Fork()) == 0)
    { /* Child runs user job */
        if (execve(argv[0], argv, environ) < 0)
        {
            printf("%s: Command not found.\n", argv[0]);
            exit(0);
        }
    }
}

```

```

/* Parent waits for foreground job to terminate */
if (!bg)
{
    int status;
    if (waitpid(pid, &status, 0) < 0)
        unix_error("waitfg: waitpid error");
}
else
    printf("%d %s", pid, cmdline);
return;
}

```

shellcmdline.c

```

/* parseline - Parse the command line and build the argv array */
int parseline(char *buf, char **argv)
{

```

```

    char *delim; /* Points to first space delimiter */
    int argc; /* Number of args */
    int bg; /* Background job? */

```

```

/* Replace trailing '\n' with space */
buf[strlen(buf)-1] = ' ';

```

```

/* Ignore leading spaces */
while (*buf && (*buf == ' '))
    buf++;

```

```

/* Build the argv list */
argc = 0;

```

```

while ((delim = strchr(buf, ' '))
{
    argv[argc++] = buf;
    *delim = '\0';
    buf = delim + 1;
    /* Ignore spaces */
    while (*buf && (*buf == ' '))
        buf++;
}
argv[argc] = NULL;

```

```

if (argc == 0) /* Ignore blank line */
    return 1;

```

```

/* Should the job run in the background? */
if ((bg = (*argv[argc-1] == '&')) != 0)
    argv[--argc] = NULL;
return bg;

```

```

/* If first arg is a builtin command, run it and
return true */

```

```

int builtin_command(char **argv)
{
    if (!strcmp(argv[0], "quit")) /* quit command */
        exit(0);
    if (!strcmp(argv[0], "&")) /* Ignore singleton & */
        return 1;
    return 0; /* Not a builtin command */
}

```

ls -la /usr/bin

buf

delim

ls\0 -la\0 /usr/bin\0

argv[0]

argv[1]

argv[2]

Using fork and execve to Run Programs

- ★ Figure shows the code that evaluates the command line.
- ★ Its first task is to call the **parseline function** which parses the space-separated command-line arguments and builds the argv vector that will eventually be passed to execve.
- ★ The first argument is assumed to be either the name of a built-in shell command that is interpreted immediately, or an executable object file that will be loaded and run in the context of a new child process.
- ★ If the last argument is an “&” character, then parseline returns 1, indicating that the program should be executed in the background (the shell does not wait for it to complete).
- ★ Otherwise it returns 0, indicating that the program should be run in the foreground (the shell waits for it to complete).

Simple Shell eval Function

```
/* eval - Evaluate a command line */
void eval(char *cmdline)
{
    /* Argument list execve() */
    char *argv[MAXARGS];
    extern char **environ;

    /* Holds modified command line */
    char buf[MAXLINE];

    int bg; /* Should the job run in bg or fg? */
    pid_t pid; /* Process id */

    strcpy(buf, cmdline);
    bg = parseline(buf, argv);

    if (argv[0] == NULL)
        return; /* Ignore empty lines */
    if (!builtin_command(argv))
    {
        if ((pid = Fork()) == 0)
        { /* Child runs user job */
            if (execve(argv[0], argv, environ) < 0)
            {
                printf("%s: Command not found.\n", argv[0]);
                exit(0);
            }
        }
    }
}
```

```
/* Parent waits for foreground job to
terminate */
if (!bg)
{
    int status;
    if (waitpid(pid, &status, 0) < 0)
        unix_error("waitfg: waitpid error");
}
else
    printf("%d %s", pid, cmdline);
}
return;
}

/* If first arg is a builtin command, run it
and return true */
int builtin_command(char **argv)
{
    if (!strcmp(argv[0], "quit")) /* quit command */
        exit(0);
    if (!strcmp(argv[0], "&")) /* Ignore singleton & */
        return 1;
    return 0; /* Not a builtin command */
}
```

Simple Shell *parseline* Function



```
/* parseline - Parse the command line and
build the argv array */

int parseline(char *buf, char **argv)
{
    char *delim; /* Points to first space delimiter */

    int argc;    /* Number of args */
    int bg;      /* Background job? */

    /* Replace trailing '\n' with space */
    buf[strlen(buf)-1] = ' ';

    /* Ignore leading spaces */
    while (*buf && (*buf == ' '))
        buf++;

    /* Build the argv list */
    argc = 0;
    while ((delim = strchr(buf, ' ')))
    {
        argv[argc++] = buf;
        *delim = '\0';
        buf = delim + 1;
        /* Ignore spaces */
        while (*buf && (*buf == ' '))
            buf++;
    }
}
```

shellcmdline.c

```
argv[argc] = NULL;
```

```
if (argc == 0) /* Ignore blank line */
    return 1;
```

```
/* Should the job run in the background? */
if ((bg = (*argv[argc-1] == '&')) != 0)
    argv[--argc] = NULL;
return bg;
```

```
}
```

parseline: Parses a line of input for the shell.

```
void unix_error(char *msg)
/* Unix-style error */
{
    fprintf(stderr, "%s: %s\n", msg,
strerror(errno));
    exit(0);
}

pid_t Fork(void)
{
    pid_t pid;

    if ((pid = fork()) < 0)
        unix_error("Fork error");

    return pid;
}
```

Using fork and execve to Run Programs

- After parsing the command line, the eval function calls the **builtin_command** function, which checks whether the first command line argument is a built-in shell command.
- If so, it interprets the command immediately and returns 1.
- Otherwise, it returns 0.
- Our simple shell has just one built-in command, the quit command, which terminates the shell.
- Real shells have numerous commands, such as pwd, jobs, and fg.
- If builtin_command returns 0, then the shell creates a child process and executes the requested program inside the child.
- If the user has asked for the program to run in the background, then the shell returns to the top of the loop and waits for the next command line.
- Otherwise the shell uses the waitpid function to wait for the job to terminate.
- When the job terminates, the shell goes on to the next iteration.

Problem with Simple Shell Example

- ★ Our example shell correctly waits for and reaps foreground jobs
- ★ Notice that this simple shell is flawed because it does not reap any of its background children.
- ★ Correcting this flaw requires the use of signals
- ★ But what about background jobs?
 - Will become zombies when they terminate
 - Will never be reaped because shell (typically) will not terminate
 - Will create a memory leak that could run the kernel out of memory

★ Solution: Exceptional control flow

- The kernel will interrupt regular processing to alert us when a background process completes
- In Unix, the alert mechanism is called a *signal*

Summary

★ Exceptions

- Events that require nonstandard control flow
- Generated externally (interrupts) or internally (traps and faults)

★ Processes

- At any given time, system has multiple active processes
- Only one can execute at a time on a single core, though
- Each process appears to have total control of processor + private memory space

Summary (cont.)

- ★ Spawning processes
 - Call `fork`
 - One call, two returns
- ★ Process completion
 - Call `exit`
 - One call, no return
- ★ Reaping and waiting for processes
 - Call `wait` or `waitpid`
- ★ Loading and running programs
 - Call `execve` (or variant)
 - One call, (normally) no return



NAME

sys/wait.h - declarations for waiting

SYNOPSIS

```
#include <sys/wait.h>
```

DESCRIPTION

The `<sys/wait.h>` header defines the following symbolic constants for use with [waitpid\(\)](#):

WNOHANG Do not hang if no status is available, return immediately.

WUNTRACED Report status of stopped child process.

and the following macros for analysis of process status values:

WEXITSTATUS() Return exit status.

WIFCONTINUED() True if child has been continued

WIFEXITED() True if child exited normally.

WIFSIGNALED() True if child exited due to uncaught signal.

WIFSTOPPED() True if child is currently stopped.

WSTOPSIG() Return signal number that caused process to stop.

WTERMSIG() Return signal number that caused process to terminate.

The following symbolic constants are defined as possible values for the *options* argument to [waitid\(\)](#):

WEXITED Wait for processes that have exited.

WSTOPPED Status will be returned for any child that has stopped upon receipt of a signal.

WCONTINUED Status will be returned for any child that was stopped and has been continued.

WNOHANG Return immediately if there are no children to wait for.

WNOWAIT Keep the process whose status is returned in *infop* in a waitable state.

The type `idtype_t` is defined as an enumeration type whose possible values include at least the following:

`P_ALL P_PID P_PGID`

The `id_t` type is defined as described in [<sys/types.h>](#).

The `siginfo_t` type is defined as described in [<signal.h>](#).

The `rusage` structure is defined as described in [<sys/resource.h>](#).

The `pid_t` type is defined as described in [<sys/types.h>](#).

Inclusion of the `<sys/wait.h>` header may also make visible all symbols from [<signal.h>](#) and [<sys/resource.h>](#).

The following are declared as functions and may also be defined as macros.

Function prototypes must be provided for use with an ISO C compiler.

`pid_t wait(int *); pid_t wait3(int *, int, struct rusage *); int waitid(idtype_t, id_t, siginfo_t *, int); pid_t waitpid(pid_t, int *, int);`

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 8 of Text Book

➤ Computer Systems : A Programmer's Perspective

- Randal E. Bryant and David R. O'Hallaron
3rd Edition, Global Edition Prentice Hall 2016
ISBN 10:1-292-10176-8**

SOC 2040

SYSTEM PROGRAMMING

WEEK 13 Lecture 1

EXCEPTIONAL CONTROL

FLOW

SIGNALS AND NONLOCAL JUMPS

EXCEPTIONAL CONTROL FLOW SIGNALS AND NONLOCAL JUMPS

LECTURE OUTLINE

- ★ **Signals**
- ★ **Nonlocal jumps**



Signals

- ★ A *signal* is a small message that notifies a process that an event of some type has occurred in the system
 - Akin to exceptions and interrupts
 - Sent from the kernel (sometimes at the request of another process) to a process
 - Signal type is identified by small integer ID's (1-31)
 - Only information in a signal is its ID and the fact that it arrived

<i>ID</i>	<i>Name</i>	<i>Default Action</i>	<i>Corresponding Event</i>
2	SIGINT	Terminate	User typed ctrl-c
4	SIGILL	Dump	Illegal Instruction
8	SIGFPE	Dump	Floating Point Exceptions
9	SIGKILL	Terminate	Kill program (cannot override or ignore)
11	SIGSEGV	Terminate	Segmentation violation
14	SIGALRM	Terminate	Timer signal
17	SIGCHLD	Ignore	Child stopped or terminate
18	SIGCONT	Continue	Resume execution, if stopped
19	SIGSTOP	Stop	Stop Process execution

#	Signal name	Default action	Comment	POSIX
1.	SIGHUP	Terminate	Hang up controlling terminal or process	Yes
2.	SIGINT	Terminate	Interrupt from keyboard	Yes
3.	SIGQUIT	Dump	Quit from keyboard	Yes
4.	SIGILL	Dump	Illegal instruction	Yes
5.	SIGTRAP	Dump	Breakpoint for debugging	No
6.	SIGABRT	Dump	Abnormal termination	Yes
6.	SIGIOT	Dump	Equivalent to SIGABRT	No
7.	SIGBUS	Dump	Bus error	No
8.	SIGFPE	Dump	Floating-point exception	Yes
9.	SIGKILL	Terminate	Forced-process termination	Yes
10.	SIGUSR1	Terminate	Available to processes	Yes
11.	SIGSEGV	Dump	Invalid memory reference	Yes
12.	SIGUSR2	Terminate	Available to processes	Yes
13.	SIGPIPE	Terminate	Write to pipe with no readers	Yes
14.	SIGALRM	Terminate	Real-timer clock	Yes
15.	SIGTERM	Terminate	Process termination	Yes
16.	SIGSTKFLT	Terminate	Coprocessor stack error	No
17.	SIGCHLD	Ignore	Child process stopped or terminated, or got signal if traced	Yes
18.	SIGCONT	Continue	Resume execution, if stopped	Yes
19.	SIGSTOP	Stop	Stop process execution	Yes
20.	SIGTSTP	Stop	Stop process issued from tty	Yes
21.	SIGTTIN	Stop	Background process requires input	Yes
22.	SIGTTOU	Stop	Background process requires output	Yes
23.	SIGURG	Ignore	Urgent condition on socket	No
24.	SIGXCPU	Dump	CPU time limit exceeded	No
25.	SIGXFSZ	Dump	File size limit exceeded	No
26.	SIGVTALRM	Terminate	Virtual timer clock	No
27.	SIGPROF	Terminate	Profile timer clock	No
28.	SIGWINCH	Ignore	Window resizing	No
29.	SIGIO	Terminate	I/O now possible	No
29	SIGPOLL	Terminate	Equivalent to SIGIO	No
30	SIGPWR	Terminate	Power Supply Failure	No
31	SIGSYS	Dump	Bad system call	No
31	SIGUNUSED	Dump	Equivalent to SIGSYS	No

Signal Concepts: Sending a Signal

- ★ Kernel *sends* (delivers) a signal to a *destination process* by updating some state in the context of the destination process
- ★ Kernel sends a signal for one of the following reasons:
 - Kernel has detected a system event such as divide-by-zero (SIGFPE) or the termination of a child process (SIGCHLD)
 - Another process has invoked the **kill** system call to explicitly request the kernel to send a signal to the destination process

Signal Concepts: Receiving a Signal

- * A destination process **receives** a signal when it is forced by the kernel to react in some way to the delivery of the signal
- * Some possible ways to react:
 - **Ignore** the signal (do nothing)
 - **Terminate** the process (with optional core dump)
 - **Catch** the signal by executing a user-level function called **signal handler**
 - * Akin to a hardware exception handler being called in response to an asynchronous interrupt:

(1) Signal received by process

I_{curr}
 I_{next}

(2) Control passes to signal handler

(3) Signal handler runs

(4) Signal handler returns to next instruction

Signal Concepts: Pending and Blocked Signals

- * A signal is **pending** if sent but not yet received
 - There can be at most one pending signal of any particular type
 - Important: Signals are not queued
 - * If a process has a pending signal of type k, then subsequent signals of type k that are sent to that process are discarded
- * A process can **block** the receipt of certain signals
 - Blocked signals can be delivered, but will not be received until the signal is unblocked
- * A pending signal is received at most once
- * For each process, the kernel maintains the set of pending signals in the pending bit vector, and the set of blocked signals in the blocked bit vector.
- * The kernel sets bit k in pending whenever a signal of type k is delivered and clears bit k in pending whenever a signal of type k is received.

Signal Concepts: Pending/Blocked Bits

- ★ Kernel maintains **pending** and **blocked** bit vectors in the context of each process
 - **pending**: represents the set of pending signals
 - ★ Kernel sets bit **k** in **pending** when a signal of type **k** is delivered
 - ★ Kernel clears bit **k** in **pending** when a signal of type **k** is received
 - **blocked**: represents the set of blocked signals
 - ★ Can be set and cleared by using the **sigprocmask** function
 - ★ Also referred to as the *signal mask*.

Sending Signals



- * Unix systems provide a number of mechanisms for sending signals to processes. All of the mechanisms rely on the notion of a process group.

❑ Process Groups

- * Every process belongs to exactly one process group, which is identified by a positive integer process groupID.
- * The `getpgrp` function returns the process group ID of the current process.

```
#include <unistd.h>
```

```
pid_t getpgrp(void);
```

Returns: process group ID of calling process

- * By default, a child process belongs to the same process group as its parent. A process can change the process group of itself or another process by using the `setpgid` function:

```
#include <unistd.h>
```

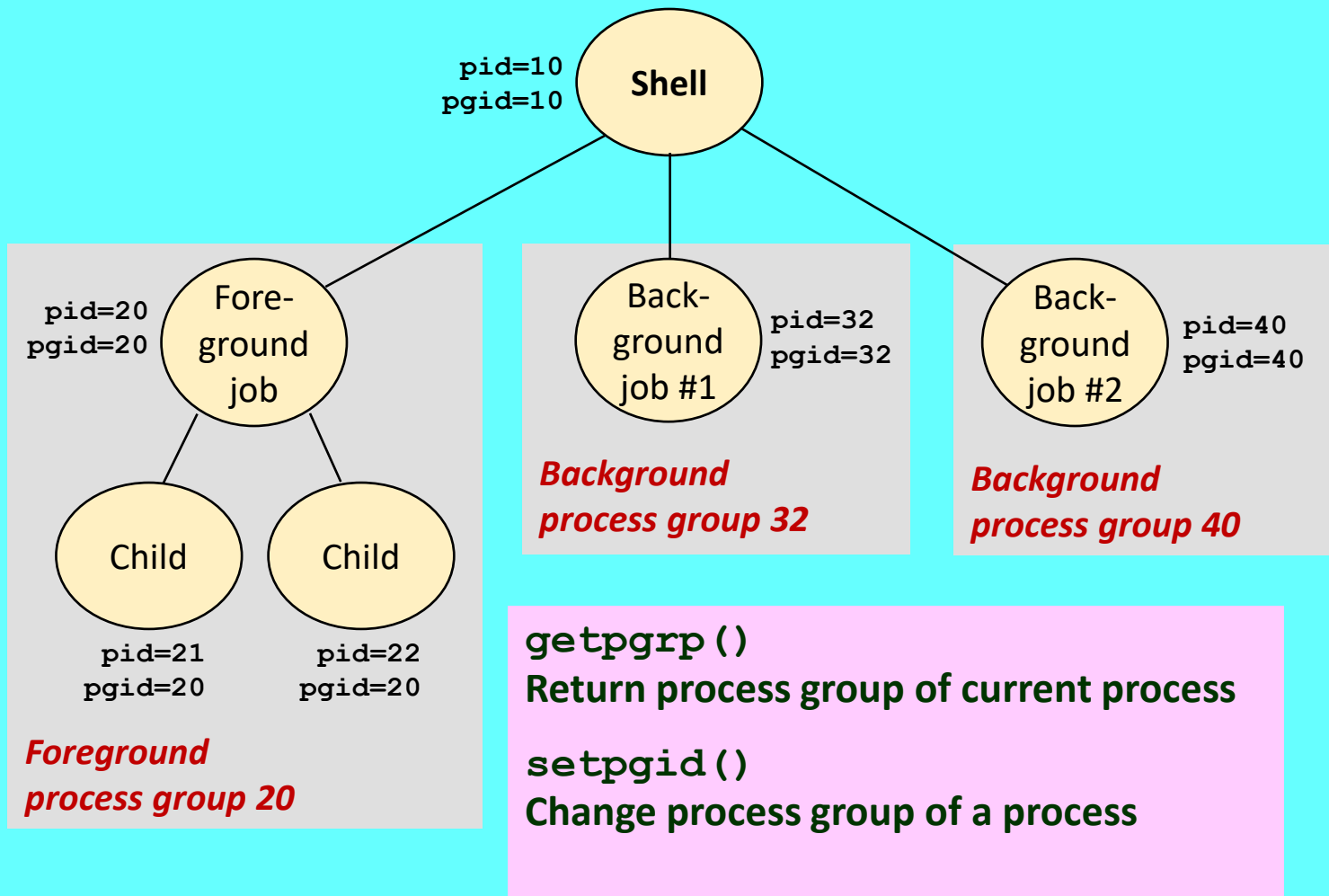
```
int setpgid(pid_t pid, pid_t pgid);
```

Returns: 0 on success, -1 on error

- * The `setpgid` function changes the process group of process `pid` to `pgid`.
- * If `pid` is zero, the PID of the current process is used.
- * If `pgid` is zero, the PID of the process specified by `pid` is used for the process groupID.
- * For example, if process 15213 is the calling process, then `setpgid(0, 0);` creates a new process group whose process group ID is 15213, and adds process 15213 to this new group.

Sending Signals: Process Groups

- ★ Every process belongs to exactly one process group



Sending Signals with `/bin/kill` Program

- * `/bin/kill` program sends arbitrary signal to a process or process group

- * Examples

- `/bin/kill -9 24818`
Send SIGKILL to process 24818
- `/bin/kill -9 -24817`
Send SIGKILL to every process in process group 24817

```
$ ./forks
```

```
Child1: pid=24818 pgrp=24817
```

```
Child2: pid=24819 pgrp=24817
```

```
$ ps
```

PID	TTY	TIME	CMD
24788	pts/2	00:00:00	tcsh
24818	pts/2	00:00:02	forks
24819	pts/2	00:00:02	forks

```
24820 pts/2 00:00:00 ps
```

```
$/bin/kill -9 -24817
```

```
$ ps
```

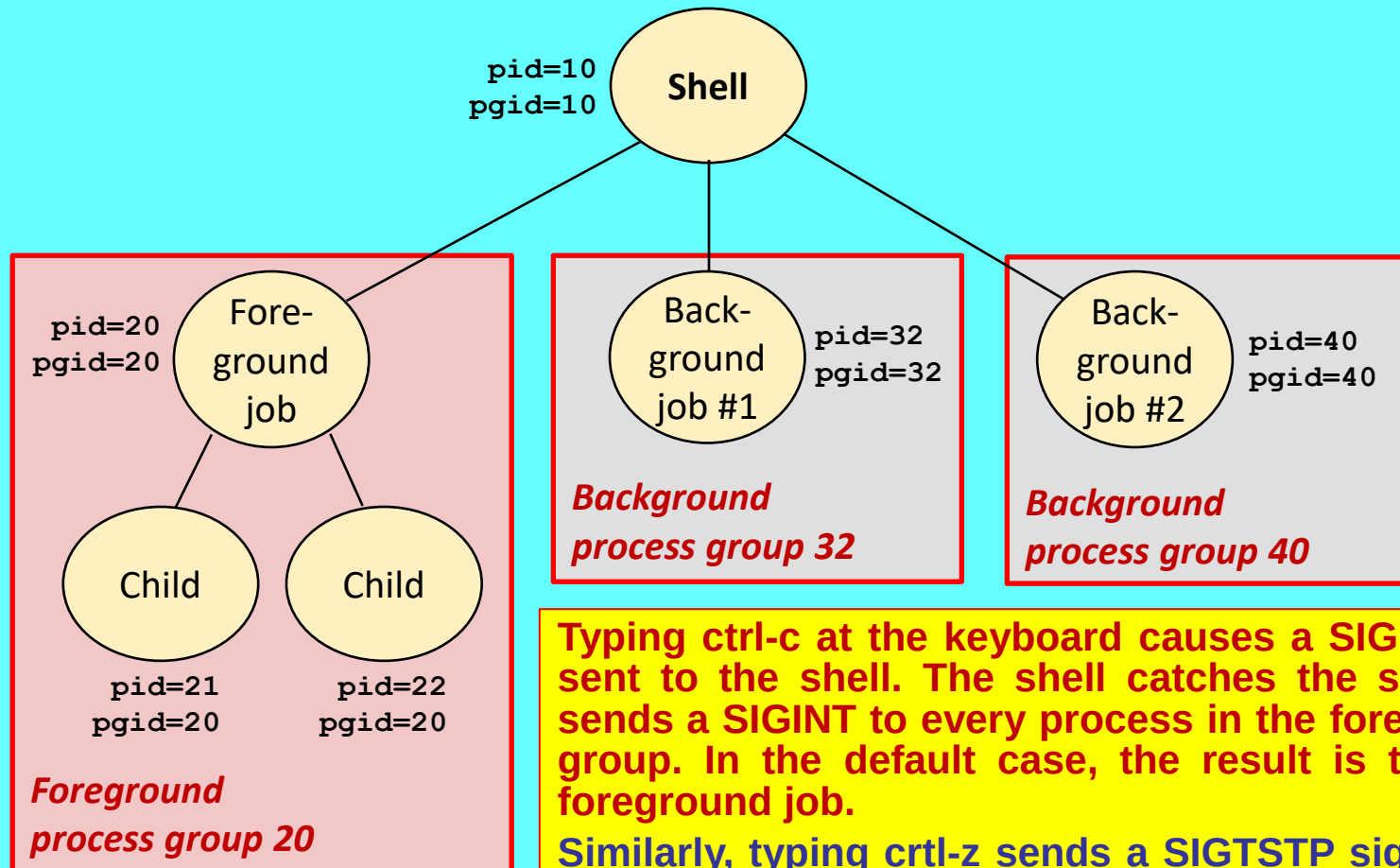
PID	TTY	TIME	CMD
24788	pts/2	00:00:00	tcsh
24823	pts/2	00:00:00	ps

```
$
```

Sending Signals from the Keyboard

The shell creates a separate process group for each job. Typically, the process group ID is taken from one of the parent processes in the job.

For example, **Figure shows a shell with one foreground job and two background jobs. The parent process in the foreground job has a PID of 20 and a process group ID of 20. The parent process has created two children, each of which are also members of process group 20.**



Typing ctrl-c at the keyboard causes a SIGINT signal to be sent to the shell. The shell catches the signal and then sends a SIGINT to every process in the foreground process group. In the default case, the result is to terminate the foreground job.

Similarly, typing ctrl-z sends a SIGTSTP signal to the shell, which catches it and sends a SIGTSTP signal to every process in the foreground process group. In the default case, the result is to stop (suspend) the foreground job.

Example of ctrl-c and ctrl-z

```
$ ./forks17
Child: pid=28108 pgrp=28107
Parent: pid=28107 pgrp=28107
<types ctrl-z>
Suspended
$ps w
  PID TTY          STAT       TIME COMMAND
27699 pts/8        Ss          0:00   -tcsh
28107 pts/8        T           0:01   ./forks17
28108 pts/8        T           0:01   ./forks17
28109 pts/8        R+          0:00   ps w
$fg
./forks 17
<types ctrl-c>
$ps w
  PID TTY          STAT       TIME COMMAND
27699 pts/8        Ss          0:00   -tcsh
28110 pts/8        R+          0:00   ps w
```

STAT (process state) Legend:

First letter:

S: sleeping

T: stopped

R: running

Second letter:

s: session leader

+: foreground proc group

Sending Signals with kill function

- * Processes send signals to other processes (including themselves) by calling the kill function.

```
#include <sys/types.h>
```

```
#include <signal.h>
```

```
int kill(pid_t pid, int sig);
```

Returns: 0 if OK, -1 on error

- * If pid is greater than zero, then the kill function sends signal number sig to process pid.
- * If pid is less than zero, then kill sends signal sig to every process in process group abs(pid).
- * Figure shows an example of a parent that uses the kill function to send a SIGKILL signal to its child.

```
int main()
```

```
{
```

```
    pid_t pid;
```

```
    /* Child sleeps until SIGKILL signal received, then dies */
```

```
    if ((pid = Fork()) == 0) {
```

```
        pause(); /* Wait for a signal to arrive */
```

```
        printf("control should never reach here!\n");
```

```
        exit(0); }
```

```
    /* Parent sends a SIGKILL signal to a child */
```

```
    kill(pid, SIGKILL);
```

```
    exit(0);
```

```
}
```

Using the kill function to send a signal to a child

Sending Signals with `kill` Function

```
void main()
{
    pid_t pid[N];
    int i;
    int child_status;

    for (i = 0; i < N; i++)
        if ((pid[i] = fork()) == 0) {
            /* Child: Infinite Loop */
            while(1)
                ;
        }

    for (i = 0; i < N; i++) {
        printf("Killing process %d\n", pid[i]);
        kill(pid[i], SIGINT);
    }

    for (i = 0; i < N; i++) {
        pid_t wpid = wait(&child_status);
        if (WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminated abnormally\n", wpid);
    }
}
```

`forks.c`



INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

14 May 2024

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG